

Conceptual CS Topics — Separate Study Required

Why this document exists: Most DSA topics get learned naturally by solving Peppcoding/LeetCode/Striver problems. But some topics are pure *conceptual knowledge* — they don't have a "problem to solve," they're explained once and asked about directly (in MCQs, interviews, or system design rounds). This document covers exactly those gaps, mapped to when they fit in your roadmap.

1. B-Trees & B+ Trees

When to study: DBMS phase (Feb-Mar 2027) — 30-60 min, conceptual only

- **What it is:** A self-balancing tree structure where each node can have multiple children (not just 2, like a binary tree) and multiple keys.
 - **Why it exists:** Databases store data on disk, and disk reads are slow. A B-tree keeps the tree very "wide and shallow" (not deep), so you reach any record in very few disk reads — this is why databases use B-trees (or B+ trees) for indexing instead of binary search trees.
 - **B-Tree vs B+ Tree:** B+ Tree stores actual data only in leaf nodes (internal nodes just have keys for navigation), and leaf nodes are linked together — this makes range queries (e.g., "find all values between X and Y") very fast. Most real databases (MySQL, PostgreSQL) use B+ Trees specifically.
 - **Key fact to remember:** A B-Tree of order m can have at most m children and $m-1$ keys per node.
 - **Source:** Search "B-Tree vs B+ Tree explained" on YouTube — a single 10-15 min video is enough.
-

2. ACID Properties (Database Transactions)

When to study: DBMS phase (Feb-Mar 2027) — 15-20 min

- **Atomicity:** A transaction either fully completes or fully fails — no partial updates (e.g., in a bank transfer, money can't leave one account without arriving in the other).
 - **Consistency:** A transaction brings the database from one valid state to another, respecting all rules/constraints.
 - **Isolation:** Concurrent transactions don't interfere with each other — as if they ran one after another, even if they actually ran simultaneously.
 - **Durability:** Once a transaction is committed, it stays saved even if the system crashes right after.
 - **Common interview question:** "Give a real example of why atomicity matters" — bank transfer is the classic answer.
-

3. Normalization (1NF, 2NF, 3NF, BCNF)

When to study: DBMS phase (Feb-Mar 2027) — 30-45 min

- **1NF:** Each column holds atomic (indivisible) values, no repeating groups (e.g., don't store "phone1, phone2" in one column — use separate rows).
 - **2NF:** 1NF + no partial dependency (every non-key column depends on the *entire* primary key, not just part of a composite key).
 - **3NF:** 2NF + no transitive dependency (a non-key column shouldn't depend on another non-key column).
 - **BCNF:** A stricter version of 3NF, handles some edge cases 3NF misses.
 - **Why it matters:** Reduces data redundancy and prevents update anomalies (e.g., updating a customer's address in one row but forgetting another row with the same customer).
-

4. CAP Theorem & Distributed Systems Basics

When to study: System Design phase (Jul-Aug 2027) — 1-2 hours

- **CAP Theorem:** A distributed system can only guarantee **2 out of 3** of: **Consistency** (all nodes see the same data at the same time), **Availability** (every request gets a response), **Partition tolerance** (system keeps working despite network failures between nodes).
 - **Why it matters:** Since network partitions *will* happen in real distributed systems, you're really choosing between Consistency and Availability when a partition occurs — this is the core tradeoff behind most database/system design decisions.
 - **Related basics to know:** Load balancing (distributing requests across servers), Caching (storing frequently-used data closer/faster), Sharding (splitting a database across multiple machines), Replication (copying data across servers for redundancy).
 - **Source:** freeCodeCamp's System Design YouTube content, or Educative's system design guide (both come up as reliable sources).
-

5. SOLID Principles (OOP Design)

When to study: OOP phase (Dec 2026-Jan 2027) — 30-45 min

- **Single Responsibility:** A class should have only one reason to change (do one thing well).
 - **Open/Closed:** Classes should be open for extension, closed for modification (add new behavior without changing existing code).
 - **Liskov Substitution:** Subclasses should be usable wherever their parent class is used, without breaking behavior.
 - **Interface Segregation:** Don't force a class to implement methods it doesn't need — prefer smaller, specific interfaces.
 - **Dependency Inversion:** Depend on abstractions (interfaces), not concrete implementations.
 - **Why it matters:** Frequently asked in OOP-design interview rounds, especially for more senior or design-heavy interviews — good to know even at your stage since it shows mature engineering thinking.
-

6. OS Concepts (Beyond Basic Process/Thread Knowledge)

When to study: OS phase (Apr 2027) — 45-60 min for these specific additions

- **Deadlock:** A situation where two or more processes are stuck waiting for each other's resources, forever. Four necessary conditions: Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait.
 - **Paging vs Segmentation:** Paging divides memory into fixed-size blocks; Segmentation divides it into variable-sized logical units (like code, stack, heap segments).
 - **CPU Scheduling algorithms:** FCFS (First Come First Serve), SJF (Shortest Job First), Round Robin, Priority Scheduling — know the basic idea of each, and that Round Robin is commonly used in time-sharing systems.
 - **Virtual Memory:** Lets a process use more memory than physically available, by swapping pages between RAM and disk.
-

7. Computer Networks — Beyond Basic Concepts

When to study: CN phase (Apr-Jun 2027) — 30-45 min for these additions

- **OSI Model (7 layers):** Physical, Data Link, Network, Transport, Session, Presentation, Application — know the order and roughly what each does (a common "name the layers" MCQ).
 - **TCP vs UDP:** TCP is connection-oriented, reliable, ordered (used for web browsing, file transfer). UDP is connectionless, faster, no guarantee of delivery/order (used for video streaming, gaming).
 - **DNS:** Translates domain names (google.com) into IP addresses.
 - **HTTP vs HTTPS:** HTTPS adds encryption (TLS/SSL) on top of HTTP for secure communication.
-

8. Union-Find / Disjoint Set Union (DSU)

When to study: Graphs phase (Apr 2027) — 1-2 days, WITH problem practice (this one you DO practice)

- **What it is:** A data structure that tracks a collection of disjoint (non-overlapping) sets, supporting two operations efficiently: `find` (which set does an element belong to) and `union` (merge two sets).
 - **Optimizations:** Path compression (flatten the tree during `find`) + Union by rank/size (attach smaller tree under bigger one) — together these give nearly $O(1)$ per operation.
 - **Common uses:** Detecting cycles in an undirected graph, counting connected components, Kruskal's Minimum Spanning Tree algorithm.
 - **Source:** Striver's Disjoint Set video (part of his graph series) — this one actually IS covered through normal problem practice, unlike the others above.
 - **Practice problems:** LeetCode "Number of Provinces," "Redundant Connection," "Accounts Merge."
-

How to Use This Document

- **Don't study all of this now.** Each section is tagged with when it fits your existing roadmap — revisit this PDF when you reach that phase.
- **Sections 1-7 are pure reading/conceptual** — a single video or article per topic is enough, no coding practice needed (except where noted).
- **Section 8 (Union-Find) is the one exception** — treat it like a normal DSA topic, with actual problem practice.
- **Total extra time investment:** roughly 4-6 hours spread across many months — genuinely small compared to your overall roadmap, just needs to not be forgotten since it won't show up automatically through problem-solving.